

Εργασία 1, Πλήρωση Τριγώνων

Στο κείμενο αναφοράς περιγράφονται όλες οι συναρτήσεις που ζητήθηκε να υλοποιηθούν, καθώς και εκείνες που χρειάστηκαν ώστε να υλοποιηθεί ο αλγόριθμος πλήρωσης των τριγώνων. Περιγράφεται ο τρόπος λειτουργίας τους, οι παράμετροι που λαμβάνουν ως είσοδο, οι έξοδοι που παράγουν, καθώς και ο τρόπος/λογική κλήσης της κάθε συνάρτησης. Οι συναρτήσεις, οι οποίες αφορούν την διαδικασία χρωματισμού των τριγώνων, παρουσιάζονται παρακάτω συνοδευόμενες και από τον αντίστοιχο κώδικα σε Python, που υλοποιήθηκε.

Συνάρτηση Γραμμικής Παρεμβολής:

Η Συνάρτηση Γραμμικής Παρεμβολής υπολογίζει την τιμή ενός διανύσματος V , το οποίο αντιστοιχεί σε ένα σημείο του χώρου (των δύο διαστάσεων) $p=(x,y)$ -με την έννοια ότι το διάνυσμα αυτό χαρακτηρίζει μια ιδιότητα του σημείου p - το οποίο ανήκει στο ευθύγραμμο τμήμα $p_1 p_2$ που ορίζουν τα σημεία $p_1=(x_1,y_1)$ και $p_2=(x_2,y_2)$ του χώρου, χρησιμοποιώντας την έννοια της γραμμικής παρεμβολής μεταξύ των διανυσμάτων V_1 και V_2 , τα οποία αντιστοιχούν στα σημεία $p_1=(x_1,y_1)$ και $p_2=(x_2,y_2)$, αντιστοίχως.

Ορίσματα/Είσοδοι της συνάρτησης:

- p_1 : Ένα διάνυσμα μήκους 2, το οποίο περιγράφει τις δισδιάστατες συντεταγμένες του σημείου/θέσης $p_1=(x_1,y_1)$, στο οποίο αντιστοιχεί το διάνυσμα V_1 . Η πρώτη συνιστώσα του διανύσματος αφορά την τετμημένη x_1 του p_1 και η δεύτερη αφορά την τεταγμένη y_1 του p_1 .
- p_2 : Ένα διάνυσμα μήκους 2, το οποίο περιγράφει τις δισδιάστατες συντεταγμένες του σημείου/θέσης $p_2=(x_2,y_2)$, στο οποίο αντιστοιχεί το διάνυσμα V_2 . Η πρώτη συνιστώσα του διανύσματος αφορά την τετμημένη x_2 του p_2 και η δεύτερη αφορά την τεταγμένη y_2 του p_2 .
- V_1 : Ένα διάνυσμα (αγνώστου) μήκους $|V_1|$ που περιγράφει τις τιμές του διανύσματος που αντιστοιχούν στη θέση p_1 .
- V_2 : Ένα διάνυσμα (αγνώστου) μήκους $|V_2|$ που περιγράφει τις τιμές του διανύσματος που αντιστοιχούν στη θέση p_2 .
- $coord$: Ένας αριθμός, ο οποίος αναπαριστά κάποια από τις δύο συντεταγμένες (x ή y) του σημείου p .
- dim : Ένας ακέραιος αριθμός. Οι “έγκυρες” τιμές του είναι 1 και 2. Αν $dim=1$, τότε η μεταβλητή $coord$ αφορά την τετμημένη x σημείου p , ενώ αν $dim=2$, τότε η μεταβλητή $coord$ αφορά την τεταγμένη y του σημείου p .

Έξοδοι της συνάρτησης:

- V : Ένα διάνυσμα (αγνώστου) μήκους $|V|$ που περιγράφει τις τιμές του διανύσματος που αντιστοιχούν στη θέση p .

Η συνάρτηση αυτή υπολογίζει το διάνυσμα V , μέσω γραμμικής παρεμβολής των διανυσμάτων V_1 και V_2 . Αυτό σημαίνει ότι ορίζει το διάνυσμα V ως εξής: $V=\lambda \cdot V_1 + (1-\lambda) \cdot V_2$, όπου μάλιστα το $\lambda \in [0,1]$. Για να μπορέσει, λοιπόν, να υλοποιηθεί η γραμμική παρεμβολή θα πρέπει να υπολογιστεί το λ . Ο τρόπος υπολογισμού της παραμέτρου λ αφορά τη θέση του σημείου p πάνω στο ευθύγραμμο τμήμα $p_1 p_2$. Αντίστοιχη σχέση ισχύει και για τα σημεία p , p_1 και p_2 , αφού το p ανήκει

στο ευθύγραμμο τμήμα $p_1 p_2$. Το λ , λοιπόν, ορίζεται ως εξής: $\lambda = \frac{norm(pp_2)}{norm(p_1 p_2)}$, όπου $norm(pp_2)$

είναι το μέτρο του ευθύγραμμου τμήματος pp_2 και αντίστοιχα $norm(p_1 p_2)$ είναι το μέτρο του ευθύγραμμου τμήματος $p_1 p_2$. Για σημεία p_1 και p_2 , τα οποία δεν ανήκουν στην ίδια οριζόντια ευθεία

μπορούμε να εφαρμόσουμε το Θεώρημα του Θαλή οπότε εύκολα αποδεικνύεται ότι:

$$\lambda = \frac{\text{norm}(pp_2)}{\text{norm}(p_1 p_2)} = \frac{y_2 - y}{y_2 - y_1} \quad (1). \text{ Αν, όμως τα σημεία } p_1 \text{ και } p_2 \text{ ανήκουν στην ίδια οριζόντια ευθεία,}$$

τότε δεν μπορεί να αξιοποιηθεί η παραπάνω σχέση, όμως εύκολα αποδεικνύεται ότι :

$$\lambda = \frac{\text{norm}(pp_2)}{\text{norm}(p_1 p_2)} = \frac{x_2 - x}{x_2 - x_1}.$$

Προφανώς, αν τα σημεία p_1 και p_2 αντιστοιχούν στο ίδιο ακριβώς σημείο (ταυτίζονται) τότε θα ισχύει, επίσης, $V = V_1 = V_2$.

Για τον υπολογισμό, λοιπόν, της παραμέτρου λ είναι αρκετό να δίνεται η τεταγμένη y σημείου p , αν τα σημεία p_1 και p_2 δεν είναι στην ίδια οριζόντια ευθεία, ενώ σε αντίθετη περίπτωση θα πρέπει να δίνεται η τεταγμένη x του p .

Η Συνάρτηση Γραμμικής Παρεμβολής υπολογίζει την τιμή της παραμέτρου λ αξιοποιώντας τη σχέση (1), όποια και αν είναι η τιμή της παραμέτρου εισόδου `dim`, δηλαδή όποια από τις δύο συντεταγμένες του p και αν δοθεί ως είσοδος `coord`. Ακόμη και αν δοθεί η τεταγμένη x τότε υπολογίζει την τεταγμένη y του p , αφού τα σημεία p , p_1 και p_2 ανήκουν στην ίδια ευθεία για την οποία μπορεί να βρεθεί ακριβώς η εξίσωσή της με δεδομένα τα σημεία p_1 και p_2 . Μόνο στην περίπτωση που τα σημεία p_1 και p_2 ανήκουν στην ίδια οριζόντια ευθεία δεν είναι χρήσιμη να δοθεί ως είσοδος η τεταγμένη y του p και αντίστοιχα στην περίπτωση που τα σημεία p_1 και p_2 ανήκουν στην ίδια κατακόρυφη ευθεία δεν είναι χρήσιμη η τεταγμένη x του p ως είσοδος.

Η Συνάρτηση Γραμμικής Παρεμβολής αξιοποιείται στη συγκεκριμένη εργασία, για τον υπολογισμό του διανύσματος RGB (τρεις χρωματικές συνιστώσες) που αντιστοιχεί σε ένα σημείο p δεδομένων των σημείων p_1 και p_2 , τα οποία ορίζουν το ευθύγραμμο τμήμα στο οποίο ανήκει το p (συνευθειακά με το p και εκατέρωθεν αυτού, δηλαδή). Η συνάρτηση αυτή θα καλείται στη συνάρτηση `g_shading` για τον υπολογισμό των χρωματικών συντεταγμένων κάθε σημείου που πρέπει να βαφτεί.

*Οι επόμενες δύο συναρτήσεις που ακολουθούν στο κείμενο της αναφοράς υλοποιήθηκαν, προκειμένου να απλουστευτούν οι αλγόριθμοι πλήρωσης τριγώνων. Θα παρουσιαστεί, εν συντομία, η λειτουργία τους και ο τρόπος κλήσης τους.

Συνάρτηση Υπολογισμού Πλευρών Τριγώνου:

Η Συνάρτηση Υπολογισμού Πλευρών Τριγώνου, έχει το όνομα `compute_lines_triangle` στα αρχεία `.py` που περιλαμβάνουν τον κώδικα και τα προγράμματα που υλοποιήθηκαν.

Ορισμός της συνάρτησης : `def compute_lines_triangle(vertices):`

Ισχύει, ότι ένα τρίγωνο ορίζεται πλήρως από τις τρεις κορυφές του, δεδομένου των συντεταγμένων τους. Η Συνάρτηση Υπολογισμού Πλευρών Τριγώνου υπολογίζει και ορίζει τις ευθείες οι οποίες διέρχονται από τις τρεις κορυφές ενός τριγώνου, το οποίο ορίζεται μέσω αυτών. Η συνάρτηση αυτή δέχεται ως είσοδο τις δισδιάστατες συντεταγμένες των τριών κορυφών ενός τριγώνου και τις τιμές των παραμέτρων m (κλίση της ευθείας) και b (σταθερός όρος) που ορίζουν την εξίσωση ευθείας : $y = m \cdot x + b$, για τις τρεις ευθείες που διέρχονται από τις κορυφές του τριγώνου (ανά δύο κάθε φορά). Αν δύο κορυφές του τριγώνου ανήκουν στην ίδια κατακόρυφη ευθεία τότε αυτό σημαίνει ότι το $m \rightarrow \infty$, το οποίο όμως δεν μπορεί να αναπαρασταθεί σε μορφή “κατανοητή” από τον υπολογιστή, οπότε σε αυτή την περίπτωση χρησιμοποιούμε τη λειτουργία της συνάρτησης `np.inf` της `numpy` βιβλιοθήκης.

Ορίσματα/Είσοδοι της συνάρτησης:

- `vertices` : Ένας πίνακας διάστασης 3×2 , που κάθε γραμμή του περιλαμβάνει τις

δισδιάστατες συντεταγμένες της κάθε κορυφής του τριγώνου. Στην πρώτη στήλη του πίνακα υπάρχουν αποθηκευμένες οι τετμημένες x των κορυφών και στη δεύτερη στήλη οι τεταγμένες y των κορυφών.

Έξοδοι της συνάρτησης:

- `triangle_lines` : Ένας πίνακας διάστασης 3×2 , του οποίου κάθε γραμμή αναφέρεται σε μια ευθεία (από τις τρεις ευθείες συνολικά που υπολογίζονται) που διέρχεται από δύο από τις τρεις κορυφές του τριγώνου. Στην πρώτη στήλη έχει την τιμή της κλίσης m (αν είναι `inf`, αυτό σημαίνει ότι το m είναι άπειρο) και στη δεύτερη στήλη έχει την τιμή του σταθερού όρου b (αν είναι `inf`, αυτό σημαίνει ότι το m είναι άπειρο).

Η Συνάρτηση Υπολογισμού Πλευρών Τριγώνου είναι ένας επαναληπτικός αλγόριθμος, στον οποίο σε κάθε επανάληψη υπολογίζεται η ευθεία (δηλαδή οι τιμές των παραμέτρων m και b που ορίζουν την ευθεία), η οποία διέρχεται από δύο διαδοχικές κορυφές του τριγώνου, όπως αυτές ορίζονται στον πίνακα `vertices`, που δίνεται σαν είσοδος. Στην πρώτη επανάληψη του αλγορίθμου υπολογίζεται η ευθεία που διέρχεται από τις κορυφές που οι συντεταγμένες τους αντιστοιχούν στην 1η και στην 2η γραμμή του πίνακα `vertices`, αντίστοιχα. Στη δεύτερη επανάληψη του αλγορίθμου υπολογίζεται η ευθεία που διέρχεται από τις κορυφές που οι συντεταγμένες τους αντιστοιχούν στην 2η και στην 3η γραμμή του πίνακα `vertices`, αντίστοιχα. Στην τελευταία (τρίτη) επανάληψη του αλγορίθμου υπολογίζεται η ευθεία που διέρχεται από τις κορυφές που οι συντεταγμένες τους αντιστοιχούν στην 1η και στην 3η γραμμή του πίνακα `vertices`, αντίστοιχα. Η σειρά αυτή του υπολογισμού έχει σημασία για τους αλγορίθμους πλήρωσης τριγώνων, όπου ο αντίστοιχος πίνακας `vertices` κάθε τριγώνου σε εκείνη την περίπτωση θα χρησιμοποιηθεί με ταξινομημένες τις κορυφές του τριγώνου (που ορίζονται στις γραμμές του πίνακα αυτού) κατά την τεταγμένη y .

Η Συνάρτηση Υπολογισμού Πλευρών Τριγώνου υπολογίζει, λοιπόν, για κάθε ζεύγος κορυφών από τον πίνακα `vertices` τις ποσότητες Dx και Dy προκειμένου να υπολογιστεί η κλίση $m = Dy/Dx$ και ο σταθερός όρος b με αντικατάσταση κάποιας από τις δύο κορυφές στην εξίσωση ευθείας $y = m \cdot x + b$. Αν οι κορυφές ανήκουν στην ίδια κατακόρυφη ευθεία τότε προφανώς $Dx = 0$ και άρα δεν μπορεί να υπολογιστεί η τιμή της μεταβλητής m και τότε γίνεται η ανάθεση $m = np.inf$, που αναφέραμε.

Η Συνάρτηση Υπολογισμού Πλευρών Τριγώνου θα χρησιμοποιηθεί στους αλγορίθμους πλήρωσης τριγώνων, αφού είναι απαραίτητο να γνωρίζουμε ποιες είναι οι πλευρές του τριγώνου, ώστε να γραφεί ο αλγόριθμος υπολογισμού των ενεργών πλευρών και ενεργών σημείων σε κάθε επανάληψη του αλγορίθμου.

Συνάρτηση Ταξινόμησης Κορυφών:

Η Συνάρτηση Ταξινόμησης Κορυφών, έχει το όνομα `sort_vertices` στα αρχεία `.py` που περιλαμβάνουν τον κώδικα και τα προγράμματα που υλοποιήθηκαν.

Ορισμός της συνάρτησης : `def sort_vertices(vertices,index):`

Η Συνάρτηση Ταξινόμησης Κορυφών λαμβάνει ως είσοδο ένα πλήθος από κορυφές, δηλαδή σημεία του χώρου (των δύο διαστάσεων), τα οποία είναι αποθηκευμένα σε έναν πίνακα δύο διαστάσεων -που στις δύο πρώτες στήλες του έχει αποθηκευμένες τις συντεταγμένες της κορυφής την οποία χαρακτηρίζει η εκάστοτε γραμμή του πίνακα. Λαμβάνει ακόμη ως είσοδο μια μεταβλητή που “δείχνει” τον τρόπο ταξινόμησης των κορυφών αυτών (κατά x ή κατά y). Η συνάρτηση αυτή αξιοποιεί, λοιπόν, τη λογική του κλασσικού αλγορίθμου ταξινόμησης αντικειμένων σε μια λίστα και επιστρέφει έναν πίνακα ίδιων διαστάσεων με αυτόν που δέχθηκε σαν είσοδο, στον οποίο είναι αποθηκευμένες οι κορυφές που ήταν και στον αρχικό πίνακα, αλλά ταξινομημένες κατά την τετμημένη τους ή την τεταγμένη τους (από τη μικρότερη προς τη μεγαλύτερη), ανάλογα με το τί δηλώνει η μεταβλητή “δείκτης”. Αν πρόκειται για ταξινόμηση κατά x , στην ουσία ο αλγόριθμος

ταξινόμησης εφαρμόζεται στα στοιχεία της πρώτης στήλης του πίνακα εισόδου και οποιαδήποτε αλλαγή (swap) μεταξύ στοιχείων της πρώτης στήλης συμβαίνει και στα αντίστοιχα στοιχεία (ίδια γραμμή πίνακα) της δεύτερης στήλης (και των υπόλοιπων στηλών του πίνακα, αν υπάρχουν, καθώς μπορεί να αφορούν κάποια ιδιότητα των σημείων-κορυφών). Όμοια, αν πρόκειται για ταξινόμηση κατά y , στην ουσία ο αλγόριθμος ταξινόμησης εφαρμόζεται στα στοιχεία της δεύτερης στήλης του πίνακα εισόδου και οποιαδήποτε αλλαγή (swap) μεταξύ στοιχείων της δεύτερης στήλης συμβαίνει και στα αντίστοιχα στοιχεία (ίδια γραμμή πίνακα) της πρώτης στήλης (και των υπόλοιπων στηλών του πίνακα, αν υπάρχουν, καθώς μπορεί να αφορούν κάποια ιδιότητα των σημείων-κορυφών).

Ορίσματα/Είσοδοι της συνάρτησης:

- **vertices** : Ένας πίνακας διάστασης $N \times M$, που κάθε γραμμή του περιλαμβάνει τις δισδιάστατες συντεταγμένες μιας κορυφής, δηλαδή ενός σημείου στις δύο πρώτες στήλες, αντίστοιχα. Στην πρώτη στήλη του πίνακα υπάρχουν αποθηκευμένες οι τετμημένες x των κορυφών και στη δεύτερη στήλη οι τεταγμένες y των κορυφών. Η διάσταση M μπορεί να είναι $M=2$, οπότε ο πίνακας περιλαμβάνει μόνο τις συντεταγμένες των σημείων, αλλά μπορεί επίσης $M>2$, οπότε ο πίνακας περιλαμβάνει και κάποιες άλλες τιμές που μπορεί να χαρακτηρίζουν την κάθε κορυφή.
- **index** : Ένας ακέραιος αριθμός, του οποίου η τιμή αποφασίζει εάν οι κορυφές θα ταξινομηθούν κατά x (αν $\text{index}=1$) ή κατά y (αν $\text{index}=2$). Οι έγκυρες τιμές είναι 1 και 2.

Έξοδοι της συνάρτησης:

- **sorted_vertices** : Ένας πίνακας διάστασης $N \times M$, του οποίου η κάθε γραμμή περιέχει τις δισδιάστατες συντεταγμένες μιας κορυφής/σημείου. Αυτός ο πίνακας περιλαμβάνει ακριβώς τις γραμμές του πίνακα **vertices** ταξινομημένες κατά x ή y (ανάλογα με την τιμή εισόδου της μεταβλητής **index**).

Η Συνάρτηση Ταξινόμησης Κορυφών θα αξιοποιηθεί στους αλγορίθμους πλήρωσης τριγώνων τόσο στη διαδικασία εύρεσης των πλευρών του τριγώνου, όσο και στη διαδικασία πλήρωσης του τριγώνου (τόσο για τον ορισμό των ακραίων τιμών για τις γραμμές σάρωσης, όσο και για την ταξινόμηση κατά x των ενεργών σημείων κάθε φορά που πρόκειται να σαρωθεί μια γραμμή σάρωσης κατά x).

Συναρτήσεις Πλήρωσης Τριγώνων:

Οι Συναρτήσεις Πλήρωσης Τριγώνων που θα παρουσιαστούν είναι δύο και είναι οι συναρτήσεις **Flat Shading** και **Gouraud Shading**. Και οι δύο αυτές συναρτήσεις βασίζονται στην υλοποίηση του αλγορίθμου πλήρωσης τριγώνων, με τη μόνη τους διαφορά να συναντάται στον τρόπο επιλογής του χρώματος, που αποδίδεται σε κάθε σημείο του “καμβά”, δηλαδή στον τρόπο χρωματισμού των τριγώνων.

Το βασικό χαρακτηριστικό, λοιπόν, των Συναρτήσεων Πλήρωσης Τριγώνων είναι ο αλγόριθμος πλήρωσης τριγώνων. Για το λόγο αυτό θα περιγράψουμε αρχικά τον αλγόριθμο αυτό, προτού εξηγηθεί η υλοποίηση του σε καθεμιά από τις συναρτήσεις **Flat Shading** και **Gouraud Shading**, με τις ανάλογες παραδοχές που κάνουμε για τον χρωματισμό των σημείων.

Ο Αλγόριθμος Πλήρωσης Τριγώνων

Ο Αλγόριθμος Πλήρωσης Τριγώνων αποτελεί μια ειδική περίπτωση του Αλγορίθμου Πλήρωσης Πολυγώνων (n -γώνων). Παρακάτω, παρουσιάζεται ο ψευδοκώδικας για τον Αλγόριθμο Πλήρωσης Πολυγώνων:

```

for k=0:1:K-1
    Βρίσκουμε τα xkmin, xkmax, ykmin, ykmax της ακμής k
end

ymin = min_k {y0min, y1min, ...}
ymax = max_k {y0max, y1max, ...}

Βρίσκουμε τη <Λίστα Ενεργών Ακμών> για την Γραμμή Σάρωσης y == ymin;
Βρίσκουμε τη <Λίστα Ενεργών Οριακών Σημείων> για την Γραμμή Σάρωσης y == ymin;

for y=ymin:1:ymax
    Διατάσσουμε τη <Λίστα Ενεργών Οριακών Σημείων> ως προς x
    cross_count=0; % Αρχικοποιούμε το πλήθος Ενεργών Οριακών Σημείων
                    % που έχουν ήδη σαρωθεί

    % Αρχή σάρωσης γραμμής y
    for x=0:1:N
        if x == τετμημένη κάποιου Ενεργού Οριακού Σημείου
            cross_count = cross_count + 1;
        end
        if (cross_count είναι ΠΕΡΙΤΤΟ)
            drawpixel(x,y)
        end
    end
    % Τέλος σάρωσης γραμμής y

    Ενημερώνουμε αναδρομικά τη <Λίστα Ενεργών Ακμών>
    Ενημερώνουμε αναδρομικά τη <Λίστα Ενεργών Οριακών Σημείων>
end

```

Εκμεταλλεούμενοι τη φιλοσοφία αυτού του αλγορίθμου, θα τον προσαρμόσουμε κατάλληλα, ώστε να προκύψει ο αντίστοιχος αλγόριθμος για την πλήρωση τριγώνων, ο οποίος θα λαμβάνει υπόψη τα ιδιαίτερα χαρακτηριστικά των τριγώνων ως πολύγωνα και θα τα αξιοποιεί για την υλοποίηση των διάφορων παραδοχών, που πρέπει να γίνουν, αλλά και την υλοποίηση των αλγορίθμων αναδρομικής ενημέρωσης της λίστας των ενεργών ακμών και της λίστας των ενεργών σημείων. Στην παρακάτω ανάλυση θα παρουσιάσουμε τον Αλγόριθμο Πλήρωσης Τριγώνων σε μορφή ψευδοκώδικα, για να αντιληφθούμε τη φιλοσοφία του και έπειτα, όπως αυτός υλοποιήθηκε σε Python.

Προτού προχωρήσουμε στη διατύπωση του αλγορίθμου, είναι απαραίτητο να σημειωθούν κάποιες βασικές ιδιότητες, που ισχύουν για κάθε τρίγωνο και θα φανούν χρήσιμες στην υλοποίηση του αλγορίθμου αυτού.

- Κάθε τρίγωνο ορίζεται πλήρως από τις τρεις κορυφές του, έστω p_1 , p_2 , p_3 , οι οποίες δίνονται με αυτή τη σειρά. Κάθε τρίγωνο διαθέτει τρεις ακμές (δηλαδή ορίζονται τρεις ευθείες που διέρχονται από τις κορυφές του τριγώνου, ανά δύο κάθε φορά) τις οποίες

ονομάζουμε e_1 (η ακμή που ενώνει τις κορυφές p_1 και p_2), e_2 (η ακμή που ενώνει τις κορυφές p_2 και p_3), e_3 (η ακμή που ενώνει τις κορυφές p_1 και p_3).

- Κάθε γραμμή σάρωσης y του grid (όπως ακριβώς ορίζεται για τον Αλγόριθμο Πλήρωσης Πολυγώνων) τέμνει το τρίγωνο σε το πολύ δύο σημεία. Μάλιστα, για τις γραμμές σάρωσης για τις οποίες ισχύει $y_{\min} \leq y \leq y_{\max}$ (όπου $y_{\min}$ και $y_{\max}$ είναι, αντίστοιχα, η μικρότερη και η μεγαλύτερη τεταγμένη από τις τεταγμένες των σημείων-κορυφών p_1, p_2, p_3 που ορίζουν το τρίγωνο), ισχύει ότι τέμνουν το τρίγωνο σε ένα σημείο (αν η γραμμή σάρωσης διέρχεται από την κορυφή με τεταγμένη $y_{\min}$ ή $y_{\max}$) ή σε δύο σημεία, που είναι σημεία των ακμών του τριγώνου. Τα σημεία τομής της γραμμής σάρωσης y με το τρίγωνο ονομάζονται ενεργά και οι ακμές στις οποίες αυτά ανήκουν, ενεργές ακμές. Φυσικά υπάρχει και η περίπτωση ένα τρίγωνο να έχει μια ακμή του οριζόντια οπότε τότε αυτή βρίσκεται ακριβώς πάνω στη γραμμή σάρωσης, αλλά όπως θα εξηγήσουμε αυτή η περίπτωση καλύπτεται από μία αντίστοιχη παραδοχή που κάνουμε για τις οριζόντιες πλευρές.
- Ένα τρίγωνο μπορεί να έχει το πολύ μια οριζόντια και το πολύ μια κατακόρυφη ακμή. Με άλλα λόγια το πολύ δύο σημεία του τριγώνου μπορεί να έχουν ίδια τεταγμένη ή ίδια τετμημένη. Με βάση αυτή την ιδιότητα, οι τιμές $y_{\min}, y_{\max}, x_{\min}, x_{\max}$ είναι καθαρά ορισμένες για κάθε τρίγωνο και αντιστοιχούν η καθεμιά μια σε μοναδικό ή σε δύο σημεία (δεν μπορούν, όμως όλες να αντιστοιχούν σε δύο σημεία).
- Αν οι κορυφές p_1, p_2, p_3 , που ορίζουν το τρίγωνο, δοθούν ταξινομημένες σε αύξουσα σειρά τιμής της τεταγμένης τους, y , τότε είναι βέβαιο ότι για τις γραμμές σάρωσης που ικανοποιούν την ιδιότητα $y_{\min} \leq y \leq y_{\max}$ (για τις οποίες εξάλλου θα υλοποιήσουμε τον αλγόριθμο, αφού όλες οι υπόλοιπες δεν τέμνουν το τρίγωνο σε κανένα σημείο του), η ακμή e_3 είναι πάντα ενεργή ακμή του τριγώνου.
- Δεδομένου ότι κάθε τρίγωνο έχει τρεις ακμές και η e_3 είναι πάντα ενεργή ακμή, τότε ο αλγόριθμος αναδρομικής ενημέρωσης των ενεργών ακμών θα “αλλάξει” τη λίστα των ενεργών ακμών το πολύ μία φορά (αναφέρω τη φράση “το πολύ”, καθώς σε περίπτωση ύπαρξης οριζόντιας ακμής στο τρίγωνο, η λίστα δεν θα “αλλάξει” σε καμία φάση εκτέλεσης του αλγορίθμου).

Παραδοχές για τον ορισμό των διαστημάτων πλήρωσης σε ένα τρίγωνο:

- Αξιοποιούμε τον κανόνα των “περιττών-άρτιων διασχίσεων”, όπως ακριβώς ορίζεται και για την πλήρωση πολυγώνων. Ορίζουμε μια μεταβλητή (εκ νέου για κάθε γραμμή σάρωσης y) που παίζει τον ρόλο ενός counter, του οποίου την τιμή αυξάνουμε κατά ένα κάθε φορά που η γραμμή σάρωσης διέρχεται από ένα ενεργό σημείο. Στην πραγματικότητα ο counter αυτός μπορεί να πάρει τιμές 0,1,2 για κάθε τρίγωνο. Κάθε φορά που η τιμή της μεταβλητής είναι περιττός, δηλαδή μόνο όταν είναι ίση με 1, τότε αποδίδουμε χρώμα στο συγκεκριμένο σημείο της γραμμής σάρωσης (βρισκόμαστε σε διάστημα πλήρωσης).
- Παραδοχή για τα ακραία σημεία, δηλαδή σημεία που ανήκουν πάνω σε ενεργές πλευρές (ακμές) του τριγώνου: Είναι γεγονός, ότι οι ενεργές πλευρές ενός τριγώνου, που φυσικά είναι και ενεργές πλευρές των γειτονικών τριγώνων στη διαδικασία σχηματισμού ενός σύνθετου αντικειμένου ή ακόμη και αν σχεδιάζουμε μόνο ένα τρίγωνο με τον υπόλοιπο “καμβά” να παραμένει λευκός (τον φανταζόμαστε να αποτελείται από τρίγωνα). Τότε, ακολουθούμε την παραδοχή ότι οι ενεργές πλευρές ανήκουν στα “δεξιότερα” και στα “πάνω” τρίγωνα. Συνεπώς, αν υπάρχει οριζόντιας ακμή στο τρίγωνο και αυτή βρίσκεται στη γραμμή σάρωσης $y=y_{\max}$, τότε δεν την θεωρούμε “μέρος” του συγκεκριμένου τριγώνου, αλλά του από “πάνω”. Επιπλέον, αν υπάρχει κατακόρυφη ακμή και αυτή βρίσκεται στην ευθεία $x=x_{\max}$, τότε δεν την θεωρούμε “μέρος” του συγκεκριμένου τριγώνου, αλλά του “δεξιότερου”. Η παραδοχή αυτή ικανοποιείται στη γενικότερη περίπτωση απόφασης αν ένα ενεργό σημείο είναι μέρος του τριγώνου, αν υλοποιήσουμε τον κανόνα των “περιττών-άρτιων διασχίσεων” με την μεταβλητή counter να αλλάζει τιμή κάθε φορά που η γραμμή σάρωσης συναντά ένα ενεργό σημείο και όχι όταν το προσπερνά, μιας και τα ενεργά σημεία είναι δύο για κάθε γραμμή σάρωσης (ακόμη και στην περίπτωση ενός ενεργού σημείου, το

οποίο αυτόματα σημαίνει ότι η γραμμή σάρωσης διασχίζει κορυφή του τριγώνου, η οποία μάλιστα δεν ανήκει σε οριζόντια ακμή, η παραδοχή αυτή έχει νόημα και αυτό θα εξηγηθεί παρακάτω).

Επιπλέον, όπως συνέβη και για τον Αλγόριθμο Πλήρωσης Πολυγώνων, κάνουμε τις παρακάτω παραδοχές για τις “οριακές” περιπτώσεις της σχέσης μιας γραμμής σάρωσης με τις ακμές του τριγώνου. Στη διαδικασία εύρεσης των διαστημάτων πλήρωσης ενός τριγώνου μπορεί να προκύψουν οι παρακάτω περιπτώσεις:

- *Γραμμές σάρωσης που διέρχονται από οριζόντιες ακμές:* Σε κάθε τρίγωνο υπάρχει το πολύ μια οριζόντια ακμή και αν για το τρίγωνο το οποίο εξετάζει ο αλγόριθμος πλήρωσης υπάρχει τέτοια, αυτή ανήκει στη γραμμή σάρωσης $y=y_{\min}$ ή στην $y=y_{\max}$. Εξ αρχής, λοιπόν, ελέγχουμε αν υπάρχει οριζόντια ακμή στο τρίγωνο και αν αυτή ανήκει στην ευθεία $y=y_{\min}$, τότε χρωματίζουμε τα σημεία της στο τέλος του αλγορίθμου (βλ. παραδοχή για οριακά σημεία).
- *Γραμμές σάρωσης που αγγίζουν κορυφές:* Όταν μια γραμμή σάρωσης τέμνει δύο διαδοχικές (μη οριζόντιες) ενεργές πλευρές (ακμές) στην κοινή κορυφή τους, τότε το διάστημα πλήρωσης έχει μηδενικό μήκος. Σε αυτή την περίπτωση, κάνουμε την παραδοχή ότι το ενεργό σημείο αυτό είναι “διπλό” και τότε ο κανόνας των “περιττών-άρτιων διασχίσεων” εφαρμόζεται κανονικά. Αυτή είναι μια απλή και εύχρηστη παραδοχή, αφού εξ αρχής γνωρίζουμε τις κορυφές του τριγώνου που θέλουμε να πληρώσουμε, άρα γνωρίζουμε τα “διπλά” ενεργά σημεία.

Με βάση όλα τα παραπάνω προχωράμε στην διατύπωση του Αλγορίθμου Πλήρωσης Τριγώνων σε ψευδοκώδικα:

Αλγόριθμος Πλήρωσης Τριγώνων

1. Διατάσσουμε τις κορυφές του τριγώνου p_1, p_2, p_3 κατά αύξουσα σειρά τεταγμένης y
2. Έστω ότι το σύνολο των κορυφών p_1, p_2, p_3 είναι ταξινομημένο κατά y , πλέον
3. Βρίσκουμε τη λίστα των ακμών $\{e_1, e_2, e_3\}$ του τριγώνου
4. Ορίζουμε τα $y_{\min}=y_1, y_{\max}=y_3$
5. Για τη γραμμή σάρωσης $y=y_{\min}$ βρίσκουμε τη λίστα των ενεργών ακμών : $\{e_1, e_3\}$
6. Για τη γραμμή σάρωσης $y=y_{\min}$ βρίσκουμε τη λίστα των ενεργών σημείων : $\{p_1, p_1\}$
7. Ονομάτισε τις ενεργές ακμές στις οποίες ανήκει το κάθε ενεργό σημείο : $\{p_1 \in e_1, p_1 \in e_3\}$
8. if (Υπάρχουν οριζόντιες ακμές στο τρίγωνο) :
9. Δημιούργησε τη λίστα `horizontal_lines` των ενεργών ακμών
10. end
11. if ($e_1 \in \text{horizontal_lines}$) :
12. $y_{\min}=y_{\min}+1$
13. Ενημέρωσε τη λίστα των ενεργών ακμών : $\{e_2, e_3\}$
14. Ενημέρωσε τη λίστα των ενεργών σημείων : Ανάλογα με το σε ποιά ακμή ανήκει το κάθε ενεργό σημείο εφάρμοσε την παρακάτω λογική για τον ορισμό του νέου:
 $-x(\text{next}) = x(\text{now}) + 1/m, y(\text{next}) = y(\text{now}) + 1$ για μη κατακόρυφες ενεργές ακμές
 $-x(\text{next}) = x(\text{now}), y(\text{next}) = y(\text{now}) + 1$ για κατακόρυφες ενεργές ακμές
15. end
16. for $y=y_{\min}:1:y_{\max}$
17. Διατάσσουμε τη λίστα των ενεργών σημείων κατά αύξουσα σειρά τετμημένης x
18. `cross_count=0`
19. # Αρχή σάρωσης της scan line y
20. for $x=0:1:N$
21. if ($x=\text{τετμημένη κάποιου ενεργού σημείου}$) :
22. `cross_count=cross_count+1`
23. end
24. if (`cross_count==περιττός`) :

25. `drawpixel(x,y)`
26. `end`
27. `end`
28. # Τέλος σάρωσης της scan line y
29. Ενημέρωσε αναδρομικά τη λίστα των ενεργών ακμών εφαρμόζοντας την παρακάτω λογική:
 - Αν $e_1 \in \text{λίστα ενεργών ακμών}$ και $y+1 \geq y_2$ (αφού οι p_1, p_2, p_3 είναι ταξινομημένες κατά y) και $e_2 \notin \text{horizontal_lines}$ τότε πρόσθεσε την στη λίστα των ενεργών ακμών και αφαίρεσε την e_1 από τη λίστα
 - Αν λίστα ενεργών ακμών == $\{e_1, e_3\}$ και $y+1 \geq y_2$ και $e_2 \in \text{horizontal_lines}$ τότε αφαίρεσε όλες τις ακμές από τη λίστα των ακμών (ο αλγόριθμος θα τερματιστεί στην επόμενη επανάληψη)
 - Σε διαφορετική περίπτωση δεν κάνουμε κάποια αλλαγή
30. Ενημέρωσε αναδρομικά τη λίστα των ενεργών σημείων εφαρμόζοντας την παρακάτω λογική:
 - Αν έχει γίνει αλλαγή στη λίστα των ενεργών ακμών από $\{e_1, e_3\}$ σε $\{e_2, e_3\}$, τότε διέγραψε το ενεργό σημείο που ανήκει στην e_1 και πρόσθεσε το p_2 , που ανήκει στην e_2 .
 - Ανάλογα με το σε ποιά ακμή ανήκει το κάθε ενεργό σημείο εφάρμοσε την παρακάτω λογική για τον ορισμό του νέου:
 - $x(\text{next}) = x(\text{now}) + 1/m$, $y(\text{next}) = y(\text{now}) + 1$ για μη κατακόρυφες ενεργές ακμές
 - $x(\text{next}) = x(\text{now})$, $y(\text{next}) = y(\text{now}) + 1$ για κατακόρυφες ενεργές ακμές
31. `end`
32. #Ελεγχε αν υπάρχει οριζόντιας ακμή που “ανήκει” στο τρίγωνο
33. `if (e1 ∈ horizontal_lines) :`
34. `for x=x1:1: x2`
35. `drawpixel(x,y1)`
36. `end`
37. `end`

Ο αλγόριθμος αυτός αξιοποιείται τόσο από την **Flat Shading** όσο και από την **Gouraud Shading**. Οι δύο συναρτήσεις διαφέρουν στον τρόπο που αποδίδουν το χρώμα σε κάθε σημείο (x,y) που επιλέγεται να χρωματιστεί, σύμφωνα με τον αλγόριθμο. Διαφέρουν, δηλαδή, ως προς την υλοποίηση της `drawpixel(x,y)`. Ο τρόπος με τον οποίο κάθε συνάρτηση “βάφει” ένα σημείο εξηγείται παρακάτω.

Στις παρακάτω εικόνες φαίνεται η υλοποίηση του Αλγορίθμου Πλήρωσης Τριγώνων σε Python. Η φωτογραφία είναι από το αρχείο `f_shading.py` οπότε το χρώμα `color_RGB` επιλέγεται με τον τρόπο που θα εξηγηθεί για την συνάρτηση **Flat Shading**. Πανομοιότυπη είναι η υλοποίηση του στο αρχείο `g_shading.py` με το χρώμα `color_RGB` να επιλέγεται εκ νέου κάθε φορά που πρέπει να χρωματιστεί ένα σημείο, όπως θα εξηγηθεί για τη συνάρτηση **Gouraud Shading**.

```
def f_shading(img,vertices,colors):
    updated_img=
    img_shape[1]
    img_shape[0]
    # Define the color has to be given to the triangle
    color_RGB=(1/3)*( np.asarray(colors[0,:]) + np.asarray(colors[1,:]) + np.asarray(colors[2,:]) )
    sorted_vertices_by_y=sort_vertices(vertices,2)

    # Compute the lines that pass through the vertices of the triangle
    # Because of sorting the vertices both the first and the last row of triangle_lines refer to the vertex with the minimum y
    # The first and the second row of triangle_lines refer to the vertex with the "median" y
    # The second and the last row of triangle_lines refer to the vertex with the maximum y
    triangle_lines=compute_lines_triangle(sorted_vertices_by_y)

    # Compute ymin and ymax for searching lines
    ymin=sorted_vertices_by_y[0,1]
    ymax=sorted_vertices_by_y[-1,1]

    # active_lines is a vector of length 3. If its i element is 1 then the i-th line is active for the searching line y.
    # active_lines will be updated during the algorithm execution.
    # active_points is an array of dimension 2x3 whose each row contains the coordinates of the point of intersection of the
    # searching line with the line included in some row of triangle_lines, at the first two columns. At the third column there
    # is the characteristic number for the active line which the active point belong to. The characteristic number is:
    # 0 for active line between vertices "1" and "2" (sorted), 1 for active line between vertices "2" and "3" (sorted),
    # 2 for active line between vertices "1" and "3" (sorted).
    # active_points will be updated during the algorithm execution.
    active_points=np.array([0,0])

    # Compute active lines for the searching line y=ymin
    active_lines=np.array([1,0,1])
```

1.

```
# Since we have triangle the number of active points for every searching line y between ymin and ymax is 2. For ymax
# and ymin there is only one active point (since y min and ymax are integers). For these cases we assume that the
# active point is double.
# Compute active points for the searching line y=ymin
active_points=np.array([ sorted_vertices_by_y[0,0],sorted_vertices_by_y[0,1], 0),
                        [sorted_vertices_by_y[-1,0],sorted_vertices_by_y[-1,1], 2] )

# Check if there are horizontal lines and save this information
horizontal_lines=np.array([0,0,0])
for i in range(3):
    if triangle_lines[i,0]==0:
        horizontal_lines[i]=1
        # If a line is horizontal, exclude it from active_lines
        active_lines[i]=0

# Special case: All the vertices are in the same horizontal line, so there is no actual triangle
# This is a special case as well as the case where the vertices are in the same vertical line but this case is contained bei
# instead of this case where (1/y)!=infinity for all the triangle lines.
if (horizontal_lines[0]==1 and horizontal_lines[1]==1) or (horizontal_lines[0]==1 and horizontal_lines[2]==1) or (horizontal
sorted_vertices_by_x=sort_vertices(sorted_vertices_by_y,0)
x1=sorted_vertices_by_x[0,0]
x2=sorted_vertices_by_x[-1,0]
for x in range(x1,x2+1):
    updated_img[x-1-x1,sorted_vertices_by_y[0,1],:]=color_RGB

return updated_img
```

2.

```
# If the line between vertices "1" and "2" (sorted) is horizontal, start scanning from ymin+1 and update active_lines and
# active_points
if horizontal_lines[0]==1:
    ymin=ymin+1
    active_lines[1]=1
    # First active point belongs to the line between vertices "1" and "3" (sorted)
    if(np.isinf(triangle_lines[2,0])!=False):
        # This is a non-vertical active line
        # Update the active point
        active_points[0,1]=ymin
        active_points[0,0]=active_points[0,0] + 1/triangle_lines[2,0]
        active_points[0,2]=2
    else:
        (variable) active_points: NDArray
        active_points[0,0]=active_points[0,0]
        active_points[0,2]=2

# Second active point belongs to the line between vertices "2" and "3" (sorted). This is active point will be calculated
# by using the vertex "2" (as initial point of this line) which belongs to the same horizontal line with vertex "1" (see
if(np.isinf(triangle_lines[1,0])!=False):
    # This is a non-vertical active line
    # Update the active point
    active_points[1,1]=ymin
    active_points[1,0]=sorted_vertices_by_y[1,0] + 1/triangle_lines[1,0]
    active_points[1,2]=1
else:
```

3.

```
else:
    # This is a vertical active line
    active_points[1,1]=ymin
    active_points[1,0]=sorted_vertices_by_y[1,0]
    active_points[1,2]=1

# Start scanning by searching lines
for y in range(ymin,ymax+1):
    # Sort active_points by x
    active_points=sort_vertices(active_points,1)
    cross_count=0
    update_now=0
    # Start scanning of search line y
    for x in range(0):
        cross_count=0
        for i in range(active_points.shape[0]):
            if x==active_points[i,0]:
                cross_count=cross_count+1
            if (cross_count % 2) !=0:
                # draw pixel
                updated_img[N-1-x,y,:]=color_RGB

# Update the list of active_lines for the next searching line

# First case: The line between vertices "1" and "2" (sorted) is active, but for the next searching line must be excluded
# 1st Sub-case: The line between vertices "2" and "3" (sorted) must be included, if and only if it is not horizontal.
```

4.

```
# First case: The line between vertices "1" and "2" (sorted) is active, but for the next searching line must be excluded
# 1st Sub-case: The line between vertices "2" and "3" (sorted) must be included, if and only if it is not horizontal.
if ysorted_vertices_by_y[1,1] and (yval)==sorted_vertices_by_y[1,1] and horizontal_lines[1]!=1:
    active_lines[0]=0
    active_lines[1]=1
    # Declare a variable to know when update the list of active_lines and which case it is.
    update_now=1
# 2nd Sub-case: The line between vertices "2" and "3" (sorted) is horizontal, which means that this line is same with
# the scanning line yval and we assume that it belongs to the triangle "above" the triangle we shade.
if ysorted_vertices_by_y[1,1] and (yval)==sorted_vertices_by_y[1,1] and horizontal_lines[1]==1:
    active_lines[0]=0
    active_lines[1]=0
    # Declare a variable to know when update the list of active_lines and which case it is.
    update_now=2

# Second case: What happens when y==ymax?
# 1st Sub-case: Searching line y has active lines both the line between vertices "1" and "2" and the line between verti-
# "1" and "3" (sorted), respectively. This means that the line between vertices "2" and "3" (sorted) is not horizontal.
if y==ymax and horizontal_lines[1]!=1:
    active_points=np.array(( sorted_vertices_by_y[2,0],sorted_vertices_by_y[2,1], 1),
        (sorted_vertices_by_y[2,0],sorted_vertices_by_y[2,1], 2) ))
    update_now=3

# 2nd Sub-case: Searching line y has active lines both the line between vertices "1" and "2" and the line between verti-
# "1" and "3" (sorted), respectively. This means that the line between vertices "2" and "3" (sorted) is horizontal. This
# case is the 2nd Sub-Case of the first case already implemented.

# Last Case: The current searching line is ymax and there is no need to update for the next searching line.
```

5.

```
# Last Case: The current searching line is ymax and there is no need to update for the next searching line
if y==ymax:
    update_now=4

# Update the list of active_points for the next searching line
if update_now==0:
    # This means no update occurred.
    # Find the active line which each active point belongs to and update the active_points for the next searching line.
    for i in range(2):
        for j in range(len(active_lines)):
            if active_lines[j]==1:
                if active_points[i,2]==j:
                    # The active line, which the active point belongs to, is found
                    if (np.isinf(triangle_lines[j,0])!=False):
                        # This means that the active line is non-vertical
                        active_points[i,0]=active_points[i,0] + 1/triangle_lines[j,0]
                        active_points[i,1]=y+1
                    else:
                        # This means that the active line is vertical
                        # Update only the y-coordinate.
                        active_points[i,1]=y+1

elif update_now==1:
    # This means that the line between vertices "2" and "3" (sorted) is now active for y+1
    # Find the active point that belongs to the line between vertices "1" and "2" (sorted), because this has to be each
    # Use the characteristic number of the point !!!
    for i in range(2):
        if(active_points[i,2]==0):
```

6.

```
for i in range(2):
    if(active_points[i,2]==0):
        # The active point, which belongs to the line between vertices "1" and "2" (sorted) and must be excluded, is
        # Include the vertex "2" (sorted) in the active points.
        active_points[i,0]=sorted_vertices_by_y[1,0]
        active_points[i,1]=sorted_vertices_by_y[1,1]
        active_points[i,2]=1
    elif(active_points[i,2]==2):
        # This active point belongs to the line between vertices "1" and "3" (sorted) and will be updated classically
        if (np.isinf(triangle_lines[2,0])!=False):
            # This means that the active line is non-vertical
            active_points[i,0]=active_points[i,0] + 1/triangle_lines[2,0]
            active_points[i,1]=y+1
        else:
            # This means that the active line is vertical
            # Update only the y-coordinate.
            active_points[i,1]=y+1

elif update_now==2:
    # In this case the line between vertices "2" and "3" is horizontal and it is assumed that belongs to the "above"
    # triangle, so the scanning process is over.
    y==ymax # This will terminate the "for" loop.
```

7.

```
elif update_now==2:
    # In this case the line between vertices "2" and "3" is horizontal and it is assumed that belongs to the "above"
    # triangle, so the scanning process is over.
    y==ymax # This will terminate the "for" loop.

# Last part of shading: Decide on horizontal (if it exists) line between two of three vertices of the triangle.
# Because of "agreement", we assume that if a horizontal line determines an "upper border" of the triangle is not
# considered to belong to the triangle (it belongs to the "above" triangle). So, if the line between vertices "1" and "2"
# (sorted) is horizontal, it is considered as part of the triangle. If the line between vertices "2" and "3" (sorted) is
# horizontal, it is not considered as part of the triangle.
# Note: Because of sorting the vertices by y, the line between vertices "1" and "3" (sorted) can't be horizontal !!!
if horizontal_lines[0]==1:
    # This means that the vertices "1" and "2" (sorted) has the same y(ymin) coordinate
    if sorted_vertices_by_y[0,0]<sorted_vertices_by_y[1,0]:
        x1=sorted_vertices_by_y[0,0]
        x2=sorted_vertices_by_y[1,0]
    else:
        x1=sorted_vertices_by_y[1,0]
        x2=sorted_vertices_by_y[0,0]
    for x in range(x1,x2):
        updated_img[N-1-x,sorted_vertices_by_y[0,1],:]=color_RGB
```

8.

Συνάρτηση Flat Shading:

Η Συνάρτηση **Flat Shading**, έχει το όνομα `f_shading` στα αρχεία `.py` που περιλαμβάνουν τον κώδικα και τα προγράμματα που υλοποιήθηκαν.

Η Συνάρτηση **Flat Shading** υλοποιεί την πλήρωση του τριγώνου, το οποίο ορίζεται από τις κορυφές που δίνονται σαν είσοδος στη συνάρτηση αυτή αποδίδοντας ένα μοναδικό χρώμα (που αντιστοιχεί σε ένα διάνυσμα `color_RGB` τριών χρωματικών συνιστωσών) σε όλα τα σημεία του τριγώνου. Το χρώμα αυτό, που όπως αναφέραμε είναι ένα διάνυσμα, προκύπτει από το διανυσματικό μέσο όρο των χρωμάτων (που επίσης είναι διανύσματα RGB τριών χρωματικών συνιστωσών) των κορυφών του τριγώνου, τα οποία δίνονται επίσης ως είσοδοι στη συνάρτηση.

Ορίσματα/Είσοδοι της συνάρτησης:

- vertices** : Ένας πίνακας διάστασης 3×2 , που κάθε γραμμή του περιλαμβάνει τις δισδιάστατες συντεταγμένες της κάθε κορυφής του τριγώνου. Στην πρώτη στήλη του πίνακα υπάρχουν αποθηκευμένες οι τετμημένες x των κορυφών και στη δεύτερη στήλη οι

τεταγμένες y των κορυφών.

- `vcolors` : Ένας πίνακας διάστασης 3×3 , που κάθε γραμμή του περιλαμβάνει τις χρωματικές συντεταγμένες (R,G,B) της αντίστοιχης κορυφής του τριγώνου που δηλώνεται στην αντίστοιχη γραμμή του πίνακα `vertices`.
- `img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που δίνεται ως είσοδος στη συνάρτηση και ενδεχομένως έχει προϋπάρχοντα χρωματισμένα τρίγωνα. Οι διαστάσεις M, N μοντελοποιούν τις διαστάσεις συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x, y) του καμβά.

Έξοδοι της συνάρτησης:

- `updated_img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που παράγεται ως έξοδος από τη συνάρτηση μετά την πλήρωση του τριγώνου που ορίζεται από τον πίνακα `vertices` πάνω στον καμβά που ορίζει ο πίνακας εισόδου `img`. Οι διαστάσεις M, N μοντελοποιούν τις διαστάσεις συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x, y) του καμβά.

Συνάρτηση Gouraud Shading:

Η Συνάρτηση **Gouraud Shading**, έχει το όνομα `g_shading` στα αρχεία `.py` που περιλαμβάνουν τον κώδικα και τα προγράμματα που υλοποιήθηκαν.

Η Συνάρτηση **Gouraud Shading** υλοποιεί την πλήρωση του τριγώνου, το οποίο ορίζεται από τις κορυφές που δίνονται σαν είσοδος στη συνάρτηση αυτή, εφαρμόζοντας τον Αλγόριθμο Πλήρωσης Τριγώνων με πανομοιότυπο τρόπο όπως και η Συνάρτηση **Flat Shading** (όπως ήδη παρουσιάστηκε παραπάνω), με τη διαφορά ότι αποδίδει χρώμα (που αντιστοιχεί σε ένα διάνυσμα `color_RGB` τριών χρωματικών συνιστωσών) σε όλα τα σημεία του τριγώνου μέσω μιας λογικής γραμμικής παρεμβολής. Αποτέλεσμα αυτού είναι όλα τα σημεία του τριγώνου να μην έχουν το ίδιο χρώμα, όπως δηλαδή συμβαίνει στη Συνάρτηση **Flat Shading**, αλλά να υπάρχουν διάφορες διαβαθμίσεις και αποχρώσεις εντός του τριγώνου. Φυσικά αυτές δεν είναι τυχαίες, αλλά προκύπτουν μέσω ενός συγκεκριμένου αλγορίθμου που αξιοποιεί μια λογική γραμμικής παρεμβολής.

Περιγραφή του αλγορίθμου χρωματισμού της Gouraud

Το χρώμα κάθε σημείου του τριγώνου, που όπως αναφέραμε είναι ένα διάνυσμα τριών χρωματικών συνιστωσών R,G,B (δηλαδή τιμές μεταξύ 0 και 1), προκύπτει -μέσω γραμμικής παρεμβολής- ως το διάνυσμα `color_RGB` που αντιστοιχεί στο σημείο αυτό, για το οποίο σημείο ισχύουν τα παρακάτω δεδομένα :

- Είναι σημείο του τριγώνου, γεγονός που σημαίνει ότι είναι εσωτερικό σημείο του τριγώνου ή σημείο κάποιας ενεργής ακμής, η οποία αποφασίζεται μέσω του Αλγορίθμου Πλήρωσης Τριγώνων ότι ανήκει στο τρίγωνο.
- Αν πρόκειται για σημείο κάποιας ενεργής ακμής, τότε πρόκειται για ενεργό σημείο του τριγώνου. Ως σημείο της ακμής του τριγώνου ισχύει ότι ανήκει στο ευθύγραμμο τμήμα που ορίζουν οι κορυφές του τριγώνου από τις οποίες ορίζεται η εν λόγω ενεργή ακμή. Σε κάθε κορυφή του τριγώνου αντιστοιχεί, επίσης, ένα διάνυσμα χρωματικών συνιστωσών το οποίο θεωρείται γνωστό. Τότε μέσω της συνάρτησης `vector_interp` με κατάλληλα ορίσματα κάθε φορά μπορεί να υπολογιστεί το διάνυσμα `color_RGB` το σημείο αυτό.
- Αν πρόκειται για σημείο στο εσωτερικό του τριγώνου, τότε ισχύει ότι το σημείο αυτό ανήκει στο ευθύγραμμο τμήμα που ορίζουν τα δύο ενεργά σημεία εκείνης της γραμμής σάρωσης y , στην οποία ανήκει επίσης το σημείο αυτό. Βρίσκοντας το χρώμα, δηλαδή το διάνυσμα χρωματικών συντεταγμένων που αντιστοιχεί σε καθένα από τα δύο αυτά ενεργά σημεία με τη λογική που περιγράφηκε στο παραπάνω bullet με χρήση της `vector_interp` με κατάλληλα

ορίσματα, εφαρμόζουμε πάλι (δεύτερη φορά) την `vector_interp` με κατάλληλα ορίσματα που αφορούν τα ενεργά σημεία αυτή τη φορά ως άκρα του ευθυγράμμου τμήματος στο οποίο ανήκει το σημείο ενδιαφέροντος και υπολογίζεται τελικά το `color_RGB` διάνυσμα.

Ο αλγόριθμος αυτός σε ψευδοκώδικα:

Αλγόριθμος Χρωματισμού Σημείου της Gouraud

1. # Σάρωση της γραμμής σάρωσης y
2. for κάθε ενεργό σημείο (2 συνολικά) της y :
3. Βρίσκουμε την ενεργή ακμή στην οποία ανήκει το ενεργό σημείο
4. υπολόγισε το διάνυσμα `active_color` του ενεργού σημείου μέσω της `vector_interp`
5. end
6. Υπολόγισε το `color_RGB` μέσω της `vector_interp` με ορίσματα τα δύο ενεργά σημεία και τα διανύσματα `active_color` που αντιστοιχούν σε αυτά

Στην παρακάτω εικόνα φαίνεται η υλοποίηση του Αλγορίθμου Χρωματισμού Σημείων της Gouraud σε Python. Η φωτογραφία είναι από το αρχείο `g_shading.py`

```
# Declare the active_colors for the searching line y
for i in range(2):
    if active_points[i,2]==0:
        # The active point belongs to the line between vertices "1" and "2" (sorted)
        if sorted_vertices_by_y[0,1]==sorted_vertices_by_y[1,1]:
            active_color1=vector_interp(sorted_vertices_by_y[0,:],sorted_vertices_by_y[1,:], vcolors_sorted_vertices[0,:],
                                      vcolors_sorted_vertices[1,:],active_points[i,0],1)
        else:
            active_color1=vector_interp(sorted_vertices_by_y[0,:],sorted_vertices_by_y[1,:], vcolors_sorted_vertices[0,:],
                                      vcolors_sorted_vertices[1,:],active_points[i,1],2)

    elif active_points[i,2]==1:
        # The active point belongs to the line between vertices "2" and "3" (sorted)
        if sorted_vertices_by_y[1,1]==sorted_vertices_by_y[2,1]:
            active_color1=vector_interp(sorted_vertices_by_y[1,:],sorted_vertices_by_y[2,:], vcolors_sorted_vertices[1,:],
                                      vcolors_sorted_vertices[2,:],active_points[i,0],1)
        else:
            active_color1=vector_interp(sorted_vertices_by_y[1,:],sorted_vertices_by_y[2,:], vcolors_sorted_vertices[1,:],
                                      vcolors_sorted_vertices[2,:],active_points[i,1],2)

    elif active_points[i,2]==2:
        # The active point belongs to the line between vertices "1" and "3" (sorted)
        if sorted_vertices_by_y[0,1]==sorted_vertices_by_y[2,1]:
            active_color1=vector_interp(sorted_vertices_by_y[0,:],sorted_vertices_by_y[2,:], vcolors_sorted_vertices[0,:],
                                      vcolors_sorted_vertices[2,:],active_points[i,0],1)
        else:
            active_color1=vector_interp(sorted_vertices_by_y[0,:],sorted_vertices_by_y[2,:], vcolors_sorted_vertices[0,:],
                                      vcolors_sorted_vertices[2,:],active_points[i,1],2)

    active_colors[i,:]=active_color1
```

Ορίσματα/Είσοδοι της συνάρτησης:

- `vertices` : Ένας πίνακας διάστασης 3×2 , που κάθε γραμμή του περιλαμβάνει τις δισδιάστατες συντεταγμένες της κάθε κορυφής του τριγώνου. Στην πρώτη στήλη του πίνακα υπάρχουν αποθηκευμένες οι τετμημένες x των κορυφών και στη δεύτερη στήλη οι τεταγμένες y των κορυφών.
- `vcolors` : Ένας πίνακας διάστασης 3×3 , που κάθε γραμμή του περιλαμβάνει τις χρωματικές συντεταγμένες (R,G,B) της αντίστοιχης κορυφής του τριγώνου που δηλώνεται στην αντίστοιχη γραμμή του πίνακα `vertices`.
- `img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που δίνεται ως είσοδος στη συνάρτηση και ενδεχομένως έχει προϋπάρχοντα χρωματισμένα τρίγωνα. Οι διαστάσεις M, N μοντελοποιούν τις δισδιάστατες συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x,y) του καμβά.

Έξοδοι της συνάρτησης:

- `updated_img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που παράγεται ως έξοδος από τη συνάρτηση μετά την πλήρωση του τριγώνου που ορίζεται από τον πίνακα `vertices` πάνω στον καμβά που ορίζει ο πίνακας εισόδου `img`. Οι διαστάσεις M, N μοντελοποιούν τις δισδιάστατες συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x,y) του καμβά.

Συνάρτηση Χρωματισμού Αντικειμένου:

Η Συνάρτηση Χρωματισμού Αντικειμένου έχει το όνομα `render_img` στα αρχεία `.py` που περιλαμβάνουν τον κώδικα και τα προγράμματα που υλοποιήθηκαν.

Η Συνάρτηση Χρωματισμού Αντικειμένου σχηματίζει ένα αντικείμενο πάνω σε έναν λευκό καμβά και αποθηκεύει το αποτέλεσμα αυτό (την εικόνα αυτή δηλαδή) σε έναν τρισδιάστατο πίνακα, οποίος αποτελεί και την έξοδο της συνάρτησης.

Η Συνάρτηση Χρωματισμού Αντικειμένου δημιουργεί έναν “καμβά” διαστάσεων $M \times N$ (όπου $M=N=512$), τον οποίο αρχικοποιεί ως λευκό καμβά. Η συνάρτηση δέχεται ως είσοδο ένα πλήθος κορυφών που ορίζουν ένα πλήθος τριγώνων στον καμβά αυτόν. Το σύνολο των τριγώνων αυτών ενδέχεται να σχηματίζουν κάποιο αντικείμενο και σε κάθε περίπτωση αποτελεί το σύνολο των τριγώνων, στα οποία θα εφαρμοστεί ο Αλγόριθμος Πλήρωσης Τριγώνων προς σχηματισμό του αντικειμένου πάνω στον λευκό καμβά. Γίνεται η υπόθεση ότι το αντικείμενο αυτό είναι ένα αντικείμενο του τρισδιάστατου χώρου, το οποίο προβάλλεται στις δύο διαστάσεις. Στο πλαίσιο αυτό, κάθε μία από τις κορυφές που δίνονται ως όρισμα χαρακτηρίζεται από έναν ακέραιο αριθμό, ο οποίος δηλώνει το βάθος της αντίστοιχης κορυφής πριν την προβολή του αντικειμένου στις δύο διαστάσεις. Ακόμη, σε κάθε μια από αυτές τις κορυφές αντιστοιχεί ένα διάνυσμα τριών χρωματικών συνιστωσών που αναπαριστά το χρώμα της κορυφής αυτής. Διαθέτοντας πίνακες για όλα αυτά τα στοιχεία, καθώς και έναν πίνακα που ορίζει τα τρίγωνα (δηλαδή ποιές κορυφές από το σύνολο των κορυφών, δηλαδή από τον πίνακα που έχει αποθηκευμένες τις δισδιάστατες συντεταγμένες κάθε κορυφής του συνόλου των δοθέντων κορυφών, σχηματίζουν κάθε τρίγωνο) η Συνάρτηση Χρωματισμού Αντικειμένου υπολογίζει έναν τρισδιάστατο πίνακα διαστάσεων $M \times N \times 3$, ο οποίος μοντελοποιεί μια εικόνα, στην οποία απεικονίζεται ο λευκός καμβάς που αρχικοποιεί η συνάρτηση και πάνω στον οποίο σχηματίζεται το αντικείμενο που δηλώνουν τα τρίγωνα που δόθηκαν ως παράμετροι εισόδου. Ο τρόπος χρωματισμού του αντικειμένου, δηλαδή καθενός από τα τρίγωνα που δομούν αυτό το αντικείμενο, καθορίζεται από μια παράμετρο εισόδου στη συνάρτηση, την μεταβλητή `shading`. Η μεταβλητή αυτή δηλώνει, αν ο τρόπος χρωματισμού επιλέγεται να είναι αυτός που ορίζει η Συνάρτηση Flat Shading ή αυτός που ορίζει η Συνάρτηση Gouraud Shading, οπότε και καλείται η ανάλογη ρουτίνα στο εσωτερικό της συνάρτησης αυτής. Επιπρόσθετα, η Συνάρτηση Χρωματισμού Αντικειμένου επιλέγει να υπολογίσει, πρώτα, τα διανύσματα που αντιστοιχούν στα χρώματα των σημείων τριγώνων με μεγαλύτερο βάθος (τρίγωνα δηλαδή που βρίσκονται πιο μακριά στο χώρο των τριών διαστάσεων) και άρα να “χρωματίσει” πρώτα τα τρίγωνα με το μεγαλύτερο βάθος, δηλαδή τα πιο απομακρυσμένα τρίγωνα στον τρισδιάστατο χώρο. Σημειώνουμε ότι το βάθος ενός τριγώνου αντιστοιχεί στο κέντρο βάρους του βάθους των τριών κορυφών που ορίζουν το τρίγωνο. Για να επιτύχει, λοιπόν, τον χρωματισμό, πρώτα, των πιο “απομακρυσμένων” τριγώνων, η Συνάρτηση Χρωματισμού Αντικειμένου υπολογίζει αρχικά ένα διάνυσμα `depth_triangles` μήκους K (αν υποθέσουμε ότι K είναι το πλήθος των τριγώνων που περιγράφονται στον πίνακα `faces` και άρα δίνονται ως είσοδος στη συνάρτηση), του οποίου το i -οστό στοιχείο αντιστοιχεί στο βάθος του τριγώνου που ορίζεται από τις τρεις κορυφές που ορίζει η i -οστή γραμμή του πίνακα `faces`. Έπειτα, η συνάρτηση ταξινομεί τον πίνακα `depth_triangles` κατά φθίνουσα σειρά τιμών. Οποιαδήποτε αλλαγή συμβαίνει σε στοιχεία του πίνακα `depth_triangles` συμβαίνει, επίσης, και στις γραμμές του πίνακα `faces` (κάθε γραμμή αφορά στις κορυφές που ορίζουν το τρίγωνο, του οποίου το βάθος αναπαρίσταται στο αντίστοιχο στοιχείο του πίνακα `depth_triangles`) αφού η i -οστή γραμμή του πίνακα `faces` αντιστοιχεί στο i -οστό στοιχείο του πίνακα `depth_triangles`, με την έννοια ότι εκείνο το στοιχείο περιέχει την πληροφορία για το βάθος του τριγώνου, άρα και μετά την ταξινόμηση του πίνακα `depth_triangles` θα πρέπει η αντιστοίχιση i -οστής γραμμής του πίνακα `faces` με το i -οστό στοιχείο του πίνακα `depth_triangles` να διατηρείται. Κατ’αυτόν τον τρόπο η Συνάρτηση Χρωματισμού Αντικειμένου επιτυγχάνει την ταξινόμηση των τριγώνων που περιγράφονται στον πίνακα `faces` κατά φθίνουσα σειρά τιμής βάθους τριγώνου. Έτσι, ανάλογα και με την τιμή της μεταβλητής `shading` (“f” ή “g”) η συνάρτηση καλεί επαναληπτικά (διατρέχοντας μία προς μία τις γραμμές του πίνακα `faces` από την 1η ως την K -οστή και άρα

ικανοποιώντας την απαίτηση για χρωματισμό πρώτα τον “μακρυνότερων” τριγώνων) την `f_shading` ή την `g_shading`, αντίστοιχα, δίνοντας ως όρισμα την τρέχουσα κατάσταση του καμβά, τις κορυφές που ορίζει ο πίνακας `faces` και των οποίων οι συντεταγμένες είναι αποθηκευμένες στον πίνακα `vertices`, καθώς και τα αντίστοιχα χρώματα των κορυφών αυτών από τον πίνακα `vcolors` και λαμβάνοντας ως έξοδο την “ανανεωμένη” κατάσταση του καμβά μετά και τον χρωματισμό του συγκεκριμένου τριγώνου.

Ορίσματα/Είσοδοι της συνάρτησης:

- `faces` : Ένας πίνακας διάστασης $K \times 3$, που κάθε γραμμή του περιλαμβάνει τις τρεις κορυφές που ορίζουν ένα τρίγωνο. Κάθε στοιχείο μια γραμμής αυτού του πίνακα είναι ένας αριθμός, από 0 έως $L-1$, που λειτουργεί σαν δείκτης στην αντίστοιχη γραμμή του πίνακα `vertices` στον οποίο αποθηκεύονται οι συντεταγμένες των κορυφών. Είναι ο πίνακας που ορίζει τα K τρίγωνα που θα χρωματιστούν.
- `vertices` : Ένας πίνακας διάστασης $L \times 2$, που κάθε γραμμή του περιλαμβάνει τις δισδιάστατες συντεταγμένες μιας κορυφής (που θεωρούμε ότι βρίσκεται εντός του καμβά). Στην πρώτη στήλη του πίνακα υπάρχουν αποθηκευμένες οι τετμημένες x των κορυφών και στη δεύτερη στήλη οι τεταγμένες y των κορυφών. Είναι ο πίνακας με τις κορυφές των τριγώνων της εικόνας, δηλαδή.
- `vcolors` : Ένας πίνακας διάστασης $L \times 3$, που κάθε γραμμή του περιλαμβάνει τις χρωματικές συντεταγμένες (R,G,B) της αντίστοιχης κορυφής του τριγώνου που δηλώνεται στην αντίστοιχη γραμμή του πίνακα `vertices`.
- `depth` : Ένας πίνακας διάστασης $L \times 1$, που κάθε στοιχείο του δηλώνει το βάθος της αντίστοιχης κορυφής (το βάθος που έχει στον τρισδιάστατο χώρο πριν την προβολή στις δύο διαστάσεις), που δηλώνεται στην αντίστοιχη γραμμή του πίνακα `vertices`.
- `shading` : Μια μεταβλητή που δηλώνει τον τρόπο χρωματισμού των τριγώνων, δηλαδή το αν θα χρησιμοποιηθεί η συνάρτηση χρωματισμού `f_shading` ή η `g_shading`. Οι έγκυρες τιμές που λαμβάνει είναι “f” (για `f_shading`) και “g” (για `g_shading`).
- `img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που δίνεται ως είσοδος στη συνάρτηση και ενδεχομένως έχει προϋπάρχοντα χρωματισμένα τρίγωνα. Οι διαστάσεις M, N μοντελοποιούν τις δισδιάστατες συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x,y) του καμβά.

Έξοδοι της συνάρτησης:

- `img` : Ένας πίνακας διάστασης $M \times N \times 3$, ο οποίος μοντελοποιεί την εικόνα που παράγεται ως έξοδος από τη συνάρτηση, δηλαδή ο λευκός καμβάς με τα K χρωματισμένα τρίγωνα που ορίζονται κατάλληλα από τους πίνακες εισόδου `faces` και `vertices`. Οι διαστάσεις M, N μοντελοποιούν τις δισδιάστατες συντεταγμένες του καμβά ενώ η τρίτη διάσταση του πίνακα αφορά το διάνυσμα τριών χρωματικών συντεταγμένων που αποδίδεται σε κάθε σημείο (x,y) του καμβά.

Scripts Επίδειξης:

Σε καθένα από τα παρακάτω δύο scripts επίδειξης, φορτώνονται τα δεδομένα που υπάρχουν στο αρχείο `hw1.npy` σε έναν πίνακα `data`, ο οποίος λειτουργεί σαν dictionary. Χειριζόμενοι το dictionary `data` κατάλληλα ορίζουμε τους πίνακες `faces`, `vertices`, `depth` και `vcolors`, οι οποίοι αποτελούν τις παραμέτρους εισόδου της συνάρτησης `render_img` σε καθένα από τα δύο scripts.

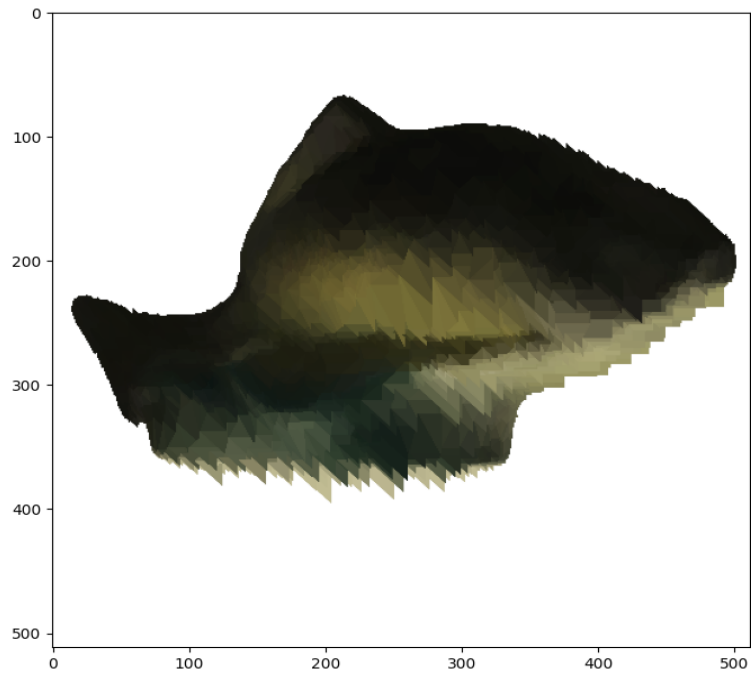
Για την κλήση της συνάρτησης `render_img` απαιτείται και η μεταβλητή “ελέγχου” του χρωματισμού `shading`. Στο script `demo_f.py` ορίζεται `shading="f"`, ενώ στο script `demo_g.py` ορίζεται `shading="g"`, προκειμένου να ακολουθηθεί ο αντίστοιχος κανόνας χρωματισμού των

τριγώνων στην τελική εικόνα.

Τέλος, σε κάθε script προβάλλεται η εικόνα που παράχθηκε (με τη χρήση της βιβλιοθήκης matplotlib) και αποθηκεύεται σαν .png (με τη χρήση της βιβλιοθήκης OpenCv).

Οι δύο εικόνες παρουσιάζονται παρακάτω.

demo_f.py



demo_g.py

